Universidad Tecnológica Nacional
Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ingeniería y Calidad de Software

TRABAJO PRÁCTICO 6 - GRUPO 10 - 4K2

TDD — Documento de decisiones de diseño
User Story: *Inscribirme a actividad*

Año: 2026

Docentes:

- Judith Meles.
- Cecilia Massano.

Integrantes:

- 94964 - CHAVES, Benjamín
- 94439 - COMBA, Valentino
- 402618 - GENERO, Cristobal
- 95312 - RIVAROLA, Matias
- 95665 - CARRAZCO CORZO, Mirco
- 95032 - CRESPIN, Sasha
- 87473 - DÍAZ, Iván
- 423349 - CARULLO PÉREZ, Lautaro
- 68160 - GUEVARA, Denis
- 94537 - MORENO MEZZADRA, Sol Fátima
- 91227 - FORMOSO, Matias Agustin
- 94180 - ZULOAGA, Sabrina Rocío

Fecha de Entrega: 19/10/2026

1. User Story implementada

COMO visitante **QUIERO** inscribirme a una actividad **PARA** reservar mi lugar en la misma. Estimación: 5 SP.

Criterios de aceptación

CA	Criterio
CA1	Seleccionar una actividad de la lista Tirolesa, Safari, Palestra y Jardinería, siempre que tenga cupos disponibles para el horario seleccionado.
CA2	Seleccionar el horario dentro de los disponibles.
CA3	Indicar la cantidad de personas que participarán.
CA4	Para cada persona: nombre, DNI, edad y talla de vestimenta si la actividad lo demanda.
CA5	Aceptar los términos y condiciones específicos de la actividad.
CA6	Al finalizar, enviar un correo con la información de la inscripción que contenga un código QR.

2. Lenguaje, herramientas y reglas de estilo

Elemento	Decisión	Justificación
Lenguaje	Python 3.12	Lenguaje elegido por el grupo; sintaxis legible para revisar las pruebas como especificación.
Framework de pruebas	pytest 9	Fixtures para preparar el contexto, `parametrize` para cubrir varios casos de borde con una sola prueba.
Estilo	PEP 8 verificado con flake8	Líneas de hasta 79 caracteres y complejidad ciclomática máxima 10 (`.flake8`). Cada commit de refactor exige flake8 sin observaciones.
Documentación	PEP 257 (docstrings)	Clases y métodos públicos documentados en una línea o un párrafo corto.
Nombres	Español, snake_case; clases en PascalCase	Mismo vocabulario que la US (visitante, actividad, cupo, talla) para que el código se lea como el dominio.
Cobertura	pytest-cov	Medición de cobertura de sentencias del paquete.

La regla de complejidad máxima tuvo un efecto concreto en el proceso: en el ciclo 8 las pruebas pasaban pero flake8 reportó complejidad 13 en `inscribir`, lo que disparó el refactor que repartió las validaciones entre las clases del modelo.

3. Supuestos del dominio

La US no fija todos los valores. Se adoptaron las siguientes reglas, agrupadas en `modelo.py` y `calendario.py` para poder ajustarlas sin tocar el servicio:

Tema	Regla adoptada
Apertura del parque	Cerrado los lunes, el 25 de diciembre y el 1 de enero.

Turnos	Las actividades duran una hora (enunciado del TP2). Turnos en hora en punto de 9 a 17 hs (el último termina a las 18). Un turno que ya comenzó no está disponible.
Cupo por turno	Tirolesa 10, Safari 8, Palestra 12, Jardinería 12.
Talla	La exigen Tirolesa y Palestra. Valores válidos: XS, S, M, L, XL, XXL.
Edad mínima	Tirolesa 8 años, Palestra 12 años; Safari y Jardinería sin mínimo.
Datos del visitante	Nombre no vacío, DNI de 7 u 8 dígitos sin puntos, edad entre 0 y 120.
Cantidad de personas	Al menos una; debe coincidir con la cantidad de visitantes cargados y no superar el cupo restante.
Correo	Se envía a la dirección de contacto informada en la inscripción, que debe tener formato válido.

4. Diseño

Módulo	Responsabilidad
`servicio.py`	`ServicioInscripcion.inscribir`: orquesta el caso de uso en pasos chicos (buscar actividad, validar horario, validar cantidad y cupo, validar participantes, términos, correo) y, si todo es válido, notifica y reserva.
`modelo.py`	`Actividad`, `Visitante`, `Inscripcion`, `Mensaje` y el catálogo de actividades.
`calendario.py`	`CalendarioParque`: días de cierre y turnos válidos.
`repositorio.py`	`RepositorioCuposEnMemoria`: lugares ocupados por actividad y horario.
`notificacion.py`	Interfaces `ServicioCorreo` y `GeneradorQR` (Protocol), validación de la dirección y armado del mensaje de confirmación.
`errores.py`	Una excepción por motivo de rechazo, todas derivadas de `InscripcionError`.
`adaptadores.py`	Implementaciones reales de las interfaces: `GeneradorQRPNG` (imagen PNG con la librería `qrcode`) y `CorreoSMTP` (envío con el QR adjunto), configurado por variables de entorno.
`web.py` y `app.py`	Demo web en Flask: formulario, mensajes de error por motivo de rechazo y pantalla de confirmación con el QR y el correo enviado.

Decisiones y justificación

Decisión	Justificación
Inyección de dependencias por constructor (cupos, correo, generador de QR, calendario, catálogo, reloj)	Permite probar el caso de uso de forma aislada, rápida y determinística, y cambiar la implementación real (base de datos, SMTP, librería de QR) sin modificar el servicio.
Dobles de prueba (`CorreoFalso`, `GeneradorQRFalso`, `CorreoCaido`)	El criterio CA6 se verifica inspeccionando el mensaje que se habría enviado, sin depender de un servidor de correo. `CorreoCaido` simula una falla de envío.
Reloj inyectable	Las pruebas de horario no dependen de la fecha en que se ejecutan: siempre usan el 20/10/2026 08:00 como "ahora".
Una excepción por criterio incumplido	Cada prueba de falla verifica el motivo exacto del rechazo y no un error genérico; la jerarquía común permite a la interfaz capturarlas todas juntas.

Experto en información	`Visitante` valida sus propios datos y `Actividad` valida si un participante cumple sus requisitos (talla y edad). El servicio no conoce esos detalles.
Objetos inmutables (`dataclass(frozen=True)`)	Una inscripción confirmada o un visitante no pueden modificarse por accidente después de validados.
Primero validar todo, después notificar y por último ocupar cupos	Una solicitud rechazada no envía correo ni reserva lugares. Si el envío del correo falla, tampoco se ocupan lugares (prueba de regresión del ciclo 12).
Catálogo como datos	Agregar una actividad o cambiar un cupo es modificar una línea del catálogo, sin agregar condicionales.
Adaptadores reales detrás de las mismas interfaces	La demo usa `CorreoSMTP` y `GeneradorQRPNG` sin cambiar una línea de `ServicioInscripcion`: confirma que la inyección de dependencias permite pasar de los dobles de prueba a la implementación real.
Credenciales SMTP en variables de entorno	Las claves no quedan en el código ni en el repositorio. Si no están definidas, la demo funciona igual y muestra en pantalla el correo que se habría enviado.
La interfaz no contiene reglas de negocio	El formulario solo convierte los datos y traduce cada excepción a un mensaje; todas las validaciones siguen en el servicio y el modelo, ya probadas.

5. Proceso TDD

Cada ciclo comenzó con una prueba que fallaba (Red), luego el código mínimo para pasarla (Green) y, cuando el diseño lo pidió, una mejora sin cambiar el comportamiento (Refactor). Cada fase quedó registrada como un commit en la rama `tp06-tdd`; un script verificó antes de cada commit que en Red hubiera pruebas fallando y que en Green y Refactor pasaran todas. Tres pruebas pasaron sin cambios de código porque el diseño ya las cubría; se dejan explícitas en el historial. Los ciclos 13 y 14 agregaron, con el mismo proceso, los adaptadores reales de correo y QR y la demo web.

Commit	Fase	Descripción
`3bdb3cb`	Base	Se crea estructura del proyecto de TP6 TDD
`8a6bc87`	Red	TDD ciclo 1 (red): se agrega prueba de inscripción exitosa
`a6597ae`	Green	TDD ciclo 1 (green): se implementa inscripción mínima
`a725a2e`	Red	TDD ciclo 2 (red): se agrega prueba de actividad fuera del listado
`a6d7495`	Green	TDD ciclo 2 (green): se valida que la actividad pertenezca al listado
`ec19e16`	Red	TDD ciclo 3 (red): se agrega prueba de actividad sin cupo para el horario
`b4d6cab`	Green	TDD ciclo 3 (green): se valida el cupo disponible por actividad y horario
`9dd580d`	Refactor	TDD ciclo 3 (refactor): se modela el catálogo de actividades con su cupo máximo
`9d7d803`	Red	TDD ciclo 4 (red): se agregan pruebas de horario con parque cerrado o actividad no disponible
`3fa4009`	Green	TDD ciclo 4 (green): se valida que el parque esté abierto y el turno exista
`6a9f099`	Refactor	TDD ciclo 4 (refactor): se extrae CalendarioParque con días de cierre y turnos
`05fb6bf`	Red	TDD ciclo 5 (red): se agrega prueba de inscripción sin aceptar términos y condiciones

`cb8d707`	Green	TDD ciclo 5 (green): se exige aceptar los términos y condiciones
`a046501`	Red	TDD ciclo 6 (red): se agregan pruebas de talle requerido por la actividad
`f844dbc`	Green	TDD ciclo 6 (green): se exige talle válido en actividades que lo requieren
`e38502b`	Aceptación	TDD ciclo 6: se agrega prueba de aceptación sin talle en actividad que no lo requiere (pasa sin cambios de código)
`d2918bc`	Red	TDD ciclo 7 (red): se agregan pruebas de cantidad de personas
`5fd847d`	Green	TDD ciclo 7 (green): se valida la cantidad de personas contra visitantes y cupo restante
`690f07a`	Red	TDD ciclo 8 (red): se agregan pruebas de datos del visitante
`5f6b077`	Green	TDD ciclo 8 (green): se validan nombre, DNI y edad de cada visitante
`927289f`	Refactor	TDD ciclo 8 (refactor): se separa en módulos y cada clase valida sus propios datos
`2de2b37`	Red	TDD ciclo 9 (red): se agregan pruebas de edad mínima por actividad
`51470ea`	Green	TDD ciclo 9 (green): se valida la edad mínima de Tirolesa y Palestra
`085c456`	Red	TDD ciclo 10 (red): se agregan pruebas de horario que ya comenzó
`26a7b8f`	Green	TDD ciclo 10 (green): se rechazan turnos que ya comenzaron usando un reloj inyectable
`8d98ebc`	Red	TDD ciclo 11 (red): se agregan pruebas del correo de confirmación con QR
`9b2ab17`	Green	TDD ciclo 11 (green): se envía correo de confirmación con código QR
`cddb5e4`	Refactor	TDD ciclo 11 (refactor): se separan las interfaces de correo y QR y los dobles de prueba
`a722e2b`	Red	TDD ciclo 12 (red): se agregan pruebas de ocupación de cupos al confirmar
`8849d8d`	Green	TDD ciclo 12 (green): se registran los lugares ocupados al confirmar la inscripción
`ccd62df`	Aceptación	TDD ciclo 12: se agrega prueba de regresión de fallo de correo sin ocupar lugares (pasa sin cambios de código)
`fdd7398`	Refactor	Refactor final: se agrupan las pruebas por prueba de usuario y criterio y se documentan las clases
`b8d5a6d`	Base	Se carga documento de decisiones de diseño del TP6 v1.0
`126c498`	Base	Se agrega carátula con docentes e integrantes al documento del TP6
`f6e9a8e`	Red	TDD ciclo 13 (red): se agregan pruebas de los adaptadores de correo SMTP y QR en PNG
`4b7560e`	Green	TDD ciclo 13 (green): se implementan CorreoSMTP, GeneradorQRPNG y la configuración por variables de entorno
`612b1c9`	Red	TDD ciclo 14 (red): se agregan pruebas de la demo web de inscripción
`3f75ce3`	Green	TDD ciclo 14 (green): se implementa la demo web con formulario, QR y vista previa del correo
`143a16e`	Corrección	Se corrige el nombre de la actividad Jardinería
`ee5d7a3`	Despliegue	Se declaran las dependencias y la configuración para publicar la demo web
`dc6da0e`	Aceptación	TDD ciclo 14: se agrega prueba del modo demostración sin variables SMTP (pasa sin cambios de código)

6. Trazabilidad con las pruebas de usuario

Prueba de usuario	Resultado esperado	Prueba automatizada
PU1: inscripción con cupo, horario, datos, talla si aplica, términos aceptados y correo	Pasa	`test_pu1_inscripcion_con_cupo_horario_datos_terminos_y_correo_pasa`
PU2: actividad sin cupo para el horario	Falla	`test_pu2_actividad_sin_cupo_para_el_horario_falla`
PU3: sin talle en actividad que no lo requiere	Pasa	`test_pu3_sin_talle_en_actividad_que_no_lo_requiere_pasa` (Safari y Jardinería)
PU4: parque cerrado o actividad no disponible en el horario	Falla	`test_pu4_horario_con_parque_cerrado_o_actividad_no_disponible_falla` (6 casos)
PU5: sin aceptar términos y condiciones	Falla	`test_pu5_sin_aceptar_terminos_y_condiciones_falla`
PU6: sin el talle requerido por la actividad	Falla	`test_pu6_sin_talle_requerido_por_la_actividad_falla` (Tirolesa y Palestra)

Los casos de borde de cada criterio están en `tests/test_reglas_de_inscripcion.py`: actividad fuera del listado, cupo exacto y excedido, ocupación de cupos, turno ya comenzado, cantidad de personas inválida, datos del visitante incompletos, talle inexistente, edad mínima, contenido del correo y del QR, correo inválido y falla en el envío.

7. Resultados y ejecución

Métrica	Valor
Pruebas automatizadas	57, todas pasan: 13 de pruebas de usuario, 32 de reglas, 6 de adaptadores y 6 de la demo web
Cobertura de sentencias	100 % del paquete `inscripcion_actividades`
Estilo	flake8 sin observaciones

Desde la carpeta `tp_06_4k2_g10_tdd_codigo_v1.0`:

```
pip install -r requirements.txt
python -m pytest
python -m pytest --cov=inscripcion_actividades --cov-report=term-missing
python -m flake8 .
```

`python app.py` (demo local en http://127.0.0.1:5000)

8. Demo web

La demo está publicada en **https://ecoharmony-inscripcion-g10.vercel.app** y se puede usar desde cualquier computadora o celular. El formulario permite elegir actividad, fecha y turno, cargar los participantes (el talle aparece solo en Tirolesa y Palestra), leer y aceptar los términos específicos e indicar el correo. Al confirmar se muestra el código de inscripción, la imagen QR y el correo de confirmación; si la solicitud no cumple una regla, se informa el motivo concreto.

Para que el correo se envíe de verdad se definen las variables de entorno `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` y opcionalmente `SMTP_FROM` (ver `.env.example`; con Gmail, `SMTP_PASS` es una contraseña de aplicación). Sin esas variables la demo trabaja en modo demostración y muestra en pantalla el correo que se habría enviado, con el QR.

9. Limitaciones y trabajo futuro

Los cupos se guardan en memoria: en la demo publicada se reinician cuando el servidor se reinicia. Quedan como trabajo futuro, detrás de la interfaz de repositorio ya definida, la persistencia en una base de datos y el control de concurrencia cuando dos visitantes reservan el último lugar al mismo tiempo.